

Tarea: Analisis amortizado

Integrantes

- Christian Echeverria 221441
- Gustavo Cruz 22779
- Josue Say 22801
- Mathew Cordero 22982
- Pedro Guzman 22111

1. Contador binario con operaciones *Increment* y *Decrement*

Worst-case scenario realista

Una secuencia alternada de operaciones *Increment* y *Decrement* que maximicen los cambios de bits.

Ejemplo concreto:

Alternar entre valores binarios con todos los bits en 1 y todos en 0 (e.g., $0111\dots 1 \iff 1000\dots 0$).

- Cada operación requiere voltear **hasta k bits** (no necesariamente todos, pero en este ejemplo sí).

Cota superior del tiempo de ejecución

- **Costo por operación:** $O(k)$ (volteo de hasta k bits).
- **Para n operaciones:** $O(nk)$.

Explicación detallada

- **Análisis tradicional (solo *Increment*):**
Costo amortizado de $O(1)$ por operación, ya que cada bit se voltea ≤ 1 vez en promedio.
- **Con *Decrement*:**
Los bits pueden cambiar repetidamente ($1 \rightarrow 0 \rightarrow 1\dots$), invalidando el análisis amortizado clásico.
- **Caso realista:**
En el ejemplo propuesto, cada operación altera k bits, resultando en $O(nk)$ operaciones totales.

2. Pila con *multipop* y *multipush*

¿Se mantiene el costo amortizado $O(1)$ por operación?

Sí, bajo las siguientes condiciones:

1. **Costo real de *multipush*(k, A):** k (equivalente a k operaciones *push* individuales).

2. **Método del potencial:**

- **Función potencial:** Φ = número de elementos en la pila.

- **Cálculo de costos amortizados:**

- ***push*:**

- Costo real: 1.

- Cambio en potencial: +1.

- Costo amortizado: $1 + 1 = 2$.

- ***multipush*(k):**

- Costo real: k .

- Cambio en potencial: $+k$.

- Costo amortizado: $k + k = 2k$.

- ***multipop*(k):**

- Costo real: $\min(k, \text{tamaño_pila})$.

- Cambio en potencial: $-k$.

- Costo amortizado: 0 (usando créditos almacenados).

Invariante clave

Cada elemento añadido “prepaga” un crédito para su eventual eliminación. Esto garantiza que:

- Los créditos acumulados en *push/multipush* cubren los costos de futuros *pop/multipop*.

- El costo amortizado por operación (individual o múltiple) sigue siendo $O(1)$.

Inciso 3

Considere una pila común y corriente cuyo tamaño nunca excede k . Luego de k operaciones *push* y/o *pop* se efectúa un *backup* automático, copiando toda la pila. El costo de copiar m elementos es m . Demuestre con el *accounting method* que cualquier secuencia de n operaciones entre *push* y *pop* (con *backups* cada k operaciones) costará $O(n)$. *Hint*: considere los costos asignados a estas operaciones en el ejemplo *multipop*.

- Se tiene una pila con capacidad máxima k .
- Cada k operaciones (*push/pop*), se realiza un *backup* automático que copia toda la pila.
- Copiar una pila con m elementos cuesta m .

- Queremos demostrar que una secuencia de n operaciones cuesta $\mathcal{O}(n)$ amortizadamente.

Solución con el Método Contable

Costos Reales

- **push**: 1
- **pop**: 1
- **backup**: m (donde m es el número de elementos actuales en la pila al momento del backup)

Costos Amortizados Propuestos

- **push**: 3 (1 por el push, 1 de crédito para futuro **pop**, 1 de crédito para backup)
- **pop**: 0 (consume crédito dejado por el **push**)
- **backup**: 0 (pagado con créditos acumulados)

Justificación de Créditos

1. Push

- Cada **push** deposita:
 - 1 crédito para su eventual **pop** (como en ejemplo **multipop**)
 - 1 crédito adicional que se usará en el backup, para pagar su costo de copia
- Como máximo hay k elementos en la pila, habrá como mucho k créditos para backup disponibles, suficientes para cubrir el costo del backup (que es $m \leq k$)

2. Pop

- Elimina un elemento de la pila
- Su costo real es 1, pero se paga con el crédito dejado por el **push**

3. Backup

- Ocurre cada k operaciones
- La pila tiene como máximo k elementos, y cada uno tiene 1 crédito reservado para backup
- Por tanto, el costo total del backup m está prepagado con los créditos acumulados

Cálculo del Costo Total Amortizado

- Cada operación **push** cuesta amortizadamente 3

- Cada **pop** y cada **backup** cuesta 0 amortizadamente
- Como hay n operaciones en total, el costo amortizado total es:

$$\mathcal{O}(n)$$

Mediante el método contable, asignando 3 unidades de costo a cada operación **push**, se logra cubrir:

- Su propio costo real (1)
- El costo de su **pop** futuro (1)
- Su participación en un **backup** futuro (1)

Así, el costo amortizado por operación es constante: $\mathcal{O}(1)$

Por tanto, el costo total de n operaciones es $\mathcal{O}(n)$.

Inciso 4

Suponga que al contador binario de los ejemplos se le agrega la operación *reset* que busca y convierte todos los 1 en 0, uno por uno a partir del bit menos significativo. Demuestre con el *accounting method* que cualquier secuencia de n operaciones entre *increment* y *reset* toma un tiempo de ejecución de $\mathcal{O}(n)$. Considere que el contador inicia desde 0 y que cada revisión y cada modificación de un bit toma $\Theta(1)$. *Hint*: ¿hasta qué bit del número binario debe llegar *reset* en cualquier momento, y cómo podemos asegurar que todos los bits que *reset* modifique tengan crédito para pagar por su reseteo?

Planetamiento

El contador hace esto nuevo

- Suma 1 con *increment*
- Vuelve todos a 0 con *reset*.

Cada revision de un bit toma de tiempo $\Theta(1)$.

Las operaciones hacen lo siguiente

Increment:

- Cambia algunos bits de 1 a 0 (los menos significativos) y un bit de 0 a 1
- Costo : $1 + \text{número de 1s consecutivos desde la derecha}$

Reset:

- Recorre desde el bit menos significativo
- Cambia los bits están en 1 a 0 osea los resetea.

- Costo: Numeros de 1s hasta la derecha

Solucion

En el accounting metod ponemos por decirlo asi creditos al costo de las operaciones.

Las operaciones de reset seran gratis esto quiere decir que operaciones anteriores pagarn su costo.

Asignar creditos:

- Si incrementamos ponemos 1 credito por incremento. Esto significa que tomaria todo lo de la cadena en cambiar osea $O(k + 1)$
- Ahora asignamos 2 creditos a increment esto porque 1 para recubrir el reseteo y 1 para el cambio.
- Para reset realmente ya lo pagamos por lo que no pagamos nada, ya que ese bit cuando fue puesto en 1. Entonces los creditos depositados que tomamos del increment los usamos.

Demostracion

El costo amortizado total es $\leq 2n$

Ahora con el metodo de accounting sabemos que

$$Costo\ Real\ Total \leq Costo\ Amortizado\ Total$$

Y como habiamos visto antes que lo que constaba cada operacion reemplazamos

$$Costo\ Amortizado\ Total \leq 2 \cdot (NumeroDeIncrement) + 0(NumeroDeReset)$$

$$Costo\ Amortizado\ Total \leq 2 \cdot (n) + 0(n)$$

$$Costo\ Amortizado\ Total \leq 2 \cdot (n)$$

Como el costo es menor a $2n$ entonces podemos decir que el costo es lineal por ende

$$O(n)$$

Esto se da gracias a que el reset llega hasta el primer 0, ya no hay más 1s consecutivos desde la derecha.

Con ello podemos asegurar que el credito de cada bit este dado, porque el increment deposita créditos (1 crédito por cada bit que pone en 1) y el reset solo los gasta

Problema 5: Función de Potencial y Costos Amortizados

Tenemos una función de potencial Φ tal que $\Phi(D_i) \geq \Phi(D_0)$ para todo i , pero $\Phi(D_0) \neq 0$. Debemos definir una función Φ' tal que: - $\Phi'(D_0) = 0$ - $\Phi'(D_i) \geq \Phi'(D_0)$ para todo $i \geq 1$ - Los costos amortizados obtenidos con Φ' sean los mismos que con Φ

Definición de la nueva función de potencial

Si definimos Φ' como:

$$\Phi'(D_i) = \Phi(D_i) - \Phi(D_0)$$

1. Condición inicial:

$$\Phi'(D_0) = \Phi(D_0) - \Phi(D_0) = 0 \quad \checkmark$$

2. Condición de no negatividad: Para todo $i \geq 1$:

$$\Phi'(D_i) = \Phi(D_i) - \Phi(D_0)$$

Como sabemos que $\Phi(D_i) \geq \Phi(D_0)$ para todo i , entonces:

$$\Phi(D_i) - \Phi(D_0) \geq 0$$

Por lo tanto: $\Phi'(D_i) \geq 0 = \Phi'(D_0)$ para todo $i \geq 1 \quad \checkmark$

3. Equivalencia de costos amortizados:

El costo amortizado de la operación i usando Φ es:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

El costo amortizado de la operación i usando Φ' es:

$$\hat{c}'_i = c_i + \Phi'(D_i) - \Phi'(D_{i-1})$$

Sustituyendo la definición de Φ' :

$$\hat{c}'_i = c_i + [\Phi(D_i) - \Phi(D_0)] - [\Phi(D_{i-1}) - \Phi(D_0)]$$

$$\hat{c}'_i = c_i + \Phi(D_i) - \Phi(D_0) - \Phi(D_{i-1}) + \Phi(D_0)$$

$$\hat{c}'_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = \hat{c}_i$$

La función $\Phi'(D_i) = \Phi(D_i) - \Phi(D_0)$ satisface todas las condiciones requeridas y proporciona una función de potencial normalizada que es equivalente a la original para el análisis de costos amortizados.

Inciso 6

Un *min-heap* binario es un árbol binario completo (todos sus niveles están llenos y sus hojas se llenan de izquierda a derecha) en donde cada nodo es menor que todos sus hijos. El *min-heap* tiene una operación de inserción llamada *insert*, pero consideremos además la función *extract-min*, que reemplaza la raíz por el último nodo del árbol y luego la intercambia con el menor de sus hijos. Supongamos que ambas operaciones tienen un tiempo de ejecución real $O(\log_2 n)$. Provea una función de potencial según la cual el costo amortizado de *insert* sea $O(\log_2 n)$ y el de *extract-min* sea $O(1)$. No olvide que el potencial siempre debe ser positivo (no necesariamente el cambio de potencial), y que el potencial inicial es idealmente cero. Demuestre que su función de potencial funciona.

Hint: primero considere la forma que tendrá el costo real de cada operación. Con esto en mente, escriba la fórmula de costo amortizado, usando valores en el cambio de potencial de forma que al sumarlo con el costo real se obtengan los costos amortizados deseados (claramente en uno de los casos el cambio de potencial debe resultar menor a $O(\log n)$; y en el otro caso debe ser negativo). ¿Qué característica(s) de un árbol binario completo se miden con un $\log_2$? ¿Cómo cambia(n) esta(s) característica(s) luego de un *insert* o un *extract-min*? Recuerde que el potencial debe extraerse de alguna propiedad de la estructura de datos.

Función de Potencial Propuesta

Para un **min-heap binario completo** con n elementos, definimos la función de potencial como:

$$\Phi(H) = \sum_{i=1}^n \log_2(\text{altura del nodo } i + 1)$$

Donde la **altura de un nodo** es la distancia máxima desde ese nodo hasta una hoja.

Propiedades de la Función de Potencial

1. $\Phi(H) \geq 0$: Siempre positiva, ya que $\log_2(\text{altura} + 1) \geq 0$ para cualquier nodo.
 2. $\Phi(\emptyset) = 0$: Un heap vacío tiene potencial cero.
 3. Extrae información estructural: El potencial refleja qué tan “desbalanceado” está el árbol.
-

Análisis de las Operaciones

Operación INSERT

Costo real:

$$C_r(\text{insert}) = \mathcal{O}(\log_2 n)$$

Cambio de potencial:

- Se añade un nuevo nodo como hoja (altura 0): $\log_2(1) = 0$
- Los nodos en el camino a la raíz pueden cambiar su contribución

$$\Delta\Phi = \mathcal{O}(\log_2 n)$$

Costo amortizado:

$$C_a(\text{insert}) = C_r + \Delta\Phi = \mathcal{O}(\log_2 n)$$

Operación EXTRACT-MIN

Costo real:

$$C_r(\text{extract-min}) = \mathcal{O}(\log_2 n)$$

Cambio de potencial:

- Eliminación de la raíz (altura $h = \lfloor \log_2 n \rfloor$):

$$\Delta\Phi = -\log_2(h+1) = -\log_2(\lfloor \log_2 n \rfloor + 1) = -\Omega(\log \log n)$$

Costo amortizado:

$$C_a(\text{extract-min}) = \mathcal{O}(\log_2 n) - \Omega(\log \log n) = \mathcal{O}(1)$$

Demostración Formal

Características del Árbol Binario Completo

- Altura total:

$$h = \lfloor \log_2 n \rfloor$$

- Número de nodos en el nivel i :

$$2^i \quad \text{para } i = 0, 1, \dots, h-1$$

- Altura de un nodo en el nivel i :

$$h - i$$

Cálculo del Potencial Total

$$\Phi(H) = \sum_{i=0}^{h-1} 2^i \cdot \log_2(h - i + 1)$$

Análisis del Extract-Min

1. Eliminación de la raíz:

$$\Delta\Phi = -\log_2(h + 1) = -\Omega(\log \log n)$$

2. Reorganización:

$$C_r = \mathcal{O}(\log_2 n)$$

3. Costo amortizado:

$$C_a = \mathcal{O}(\log_2 n) - \Omega(\log \log n) = \mathcal{O}(1)$$

Validación de la Función de Potencial

- Captura la estructura: nodos más altos contribuyen más al potencial.
 - **extract-min** libera suficiente potencial al eliminar la raíz.
 - **insert** añade potencial gradualmente.
 - Se mantiene la correlación entre potencial y “costo estructural” del heap.
-

Conclusión

La función de potencial:

$$\Phi(H) = \sum_{i=1}^n \log_2(\text{altura del nodo } i + 1)$$

permite demostrar que:

- **INSERT**: $C_a = \mathcal{O}(\log_2 n)$
- **EXTRACT-MIN**: $C_a = \mathcal{O}(1)$

Esto se logra porque **extract-min** libera suficiente potencial acumulado (al eliminar la raíz del heap) para compensar su costo real de $\mathcal{O}(\log_2 n)$.